ML Defender (aRGus NDR): An Open-Source Embedded ML NIDS for Botnet and Anomalous Traffic Detection in Resource-Constrained Organizations

Alonso Isidoro Román
Independent Researcher, Extremadura, Spain
github.com/alonsoir/argus

Draft v11 — April 2026

Abstract

Ransomware and DDoS attacks disproportionately impact hospitals, schools, and small organizations that cannot afford enterprise security solutions. Existing open-source alternatives either rely on signature matching — missing novel variants — or require infrastructure beyond the reach of their target users. We present **ML Defender (aRGus NDR)**, an open-source network intrusion detection system built in C++20 that brings embedded machine learning inference to organizations with limited resources, deployable on commodity bare-metal hardware at approximately 150–200 USD fully dedicated.

ML Defender implements a six-component pipeline over eBPF/XDP packet capture, ZeroMQ transport, and Protocol Buffers serialization, terminating in a dual-score detection architecture combining a rule-based Fast Detector with an embedded Random Forest classifier. The *Maximum Threat Wins* policy selects the arithmetic maximum of both detector scores, using ML inference to suppress false positives generated by the heuristic layer.

We evaluate ML Defender against the CTU-13 Neris botnet dataset, achieving F1 = 0.9985, Precision = 0.9969, and Recall = 1.0000, with a false positive rate of 0.0002% (2 FP in 12,075 benign flows) — the two false positives identified as host-only adapter artifacts: an mDNS multicast frame (192.168.56.1 → 224.0.0.251) and a broadcast frame (192.168.56.1 → 192.168.56.255), both generated by the VirtualBox host-only NIC and absent from production network topologies. Whether these artifacts appear in bare-metal deployments remains unverified; bare-metal characterization is Future Work (§11). The Fast Detector alone produces a FPR of 6.61% on purely benign traffic (bigFlows); the ML Detector reduces real production blocks to zero on the same corpus, an approximately 500-fold reduction. Per-class inference latency ranges from 0.24 μs (DDoS) to 1.06 μs (Ransomware), enabling real-time detection on commodity hardware.

We further characterize the pipeline's throughput ceiling under progressive load in a virtualized environment. Requesting up to 100 Mbps via `tcpreplay` against the VirtualBox NIC, the pipeline processes up to ∼34–38 Mbps (the emulated NIC ceiling) with zero packet drops and zero errors across 2.37 million packets. At peak load, `ml-detector` consumes approximately 3.2 CPU cores; total pipeline RAM remains stable at ∼1.28 GB with negligible drift (∼18 MB over 8 minutes). The bottleneck is VirtualBox NIC emulation, not pipeline

logic. Post-replay drain behavior confirms that ZeroMQ queues absorb burst traffic correctly and drain asynchronously, validating the queue stability design. All throughput figures should be treated as conservative lower bounds; bare-metal characterization remains future work (§11).

These results demonstrate architectural feasibility under controlled replay conditions rather than universal detection capability. The evaluation covers a single botnet scenario from 2011 with synthetic-trained classifiers; generalizability to modern ransomware, DDoS variants, and encrypted C2 traffic has not been empirically established and is discussed in Section 10. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the Neris scenario; direct evaluation against post-2020 ransomware families remains Future Work §11.

All experiments were conducted in a reproducible Vagrant/VirtualBox environment running on a consumer laptop, ensuring result accessibility and independent reproducibility.

This work was developed through a structured multi-model AI collaboration methodology — the *Consejo de Sabios* (Council of Wise Men), a multi-LLM peer review methodology — documented in Section 6. *Test-Driven Hardening* (TDH) is proposed as a methodology for building security-critical distributed systems, emerging directly from this collaboration. ML Defender is released under the MIT license.

Contents

1 Introduction **5**

2 Background and Related Work **6**

3 Threat Model **7**
- 3.1 Adversary Capabilities . . . 7
- 3.2 Adversary Limitations . . . 7
- 3.3 Detection Surface . . . 7
- 3.4 Out-of-Scope Attacks . . . 7

4 Architecture **7**
- 4.1 Pipeline Overview . . . 7
- 4.2 Three Functions of the Engine . . . 8
- 4.3 Dual-Score Detection . . . 8
- 4.4 Transport and Serialization . . . 9
- 4.5 Deployment Model . . . 9
- 4.6 Service Coordination (etcd-server) . . . 9
- 4.7 Fast Detector Heuristics (DEBT-FD-001) . . . 10

5 Implementation **10**
- 5.1 Language and Build System . . . 10
- 5.2 eBPF/XDP Packet Capture . . . 10
- 5.3 Feature Extraction . . . 10
- 5.4 Embedded Random Forest and Synthetic Training Data . . . 10
- 5.5 Cryptographic Transport . . . 11

5.6 Plugin Architecture (ADR-023, ADR-024) . . . 11
5.7 Trace Correlation . . . 12
5.8 Testing and Validation . . . 12
5.9 Development Environment . . . 12

6 The Consejo de Sabios: A Methodology for Human-AI Collaborative Engineering 12
6.1 Motivation . . . 12
6.2 The Methodology . . . 12
6.3 How It Worked in Practice . . . 12
6.4 Test Driven Hardening . . . 13
6.5 HKDF Context Symmetry: A Pedagogical Case Study in Test-Driven Hardening 13
6.6 Attribution . . . 15
6.7 Why This Matters . . . 15
6.8 Limitations . . . 15

7 Formal System Model 15
7.1 Network Traffic Representation . . . 15
7.2 Feature Extraction . . . 15
7.3 Fast Detector . . . 15
7.4 Machine Learning Detector . . . 16
7.5 Detection Policy . . . 16
7.6 Response Function . . . 16
7.7 System Determinism . . . 16

8 Evaluation 16
8.1 Experimental Setup . . . 16
8.2 Dataset . . . 17
8.3 Detection Performance . . . 17
8.4 False Positive Identification and Confusion Matrix . . . 17
8.5 Inference Latency . . . 18
8.6 Ablation Study . . . 18
8.7 Interpretation and Comparison with State of the Art . . . 18
8.8 Ransomware Detection: Scope and Validation . . . 19
8.9 Throughput Stress Test and Resource Utilization . . . 19

9 Performance Model and Throughput Analysis 20
9.1 Pipeline Processing Model . . . 20
9.2 Inference Cost . . . 20
9.3 Throughput: Theoretical Bound and Measured Ceiling . . . 20
9.4 Queue Stability . . . 20
9.5 Scalability . . . 20

10 Limitations 20

11 Future Work **21**

12 Conclusion **22**

13 Reproducibility and Artifact Availability **24**

1 Introduction

The threat is not abstract. On March 5, 2023, the Hospital Clínic de Barcelona suffered a ransomware attack that paralyzed its emergency services, laboratory, and pharmacy, encrypted 4.5 TB of patient data, and prompted a ransom demand of 4.5 million USD — which the institution refused to pay [6]. That same day, the author of this work was leaving a regional hospital in Extremadura after receiving treatment for porphyria — a rare metabolic disease that his local hospital had diagnosed and managed. Watching the news, the thought was immediate and personal: *if it can happen to a major teaching hospital, it can happen to mine.*

This was not the first time that the author had witnessed the human cost of ransomware firsthand. Years earlier, the collapse of a close friend's small business had been caused by an encryption attack that locked every document and file in his office. The author lacked the technical skills to help. That failure left a debt.

These two events — the friend's destroyed business, the hospital on television — converged into a single design constraint: the solution had to be open-source, it had to run on commodity hardware, and it had to act in real time. Not forensics. Not post-mortem analysis. Interception.

The analogy that emerged from this motivation is biological. Network traffic behaves like a circulatory system: datagrams are the bloodstream, and malicious flows are pathogens. A useful defense must classify traffic continuously and in real time — capturing the source IP of the attacker and blocking it at the firewall before damage propagates. This framing shaped every architectural decision in ML Defender.

This work therefore addresses the following research question: *can an embedded machine-learning network detection and response system running entirely on commodity hardware provide meaningful real-time protection for organizations lacking enterprise security infrastructure?*

A note on terminology. ML Defender (aRGus NDR) implements *Network Detection and Response* (NDR) capabilities: real-time capture, ML-based classification, and automated blocking at the network layer. Future work toward full *Endpoint Detection and Response* (EDR) functionality via a planned lightweight endpoint agent (§11, FEAT-EDR-1) will be reflected in a corresponding rename when that capability is delivered.

The gap. Existing open-source NIDS — most notably Snort and Suricata [9] — operate primarily on signature matching and rule-based heuristics. While effective against known attack patterns, they offer limited resilience against novel ransomware variants and volumetric DDoS campaigns. Machine learning-based NIDS have been extensively studied [3, 10], but the gap between research prototype and deployable system remains wide: most published systems are evaluated on benchmark datasets without accompanying implementation, require GPU infrastructure, or assume dedicated security operations teams. Healthcare retained its position as the most targeted sector in 2025, accounting for 22% of disclosed ransomware attacks, with an average breach cost of 7.42M USD [2, 5].

Our approach. ML Defender (aRGus NDR) addresses this gap with three design principles: (1) *embedded inference* — all ML classification runs in-process at sub-microsecond latency;

(2) *commodity deployment* — the full six-component pipeline runs on bare-metal Linux hardware costing approximately 150–200 USD; (3) *active response* — upon detection, the attacker's IP is blocked immediately at the firewall via ipset/iptables.

Beyond the technical system, this paper documents a second contribution: the *Consejo de Sabios* methodology for human-AI collaborative engineering.

2 Background and Related Work

Signature-based NIDS. Snort [11] and Suricata [9] apply rule-based pattern matching against packet payloads and flow metadata. Their fundamental limitation is reactive: detection requires a prior known signature, a window that historically ranges from hours to weeks.

ML-based NIDS. Anderson & McGrew [1] demonstrated the feasibility of contextual flow data for encrypted malware classification. Mirsky et al. [8] proposed Kitsune, an ensemble of autoencoders for unsupervised anomaly detection operating on flow statistics. The CTU-13 dataset [4], which ML Defender uses for evaluation, has become a standard benchmark for botnet traffic detection. Despite this body of work, a persistent gap exists between academic prototype and production system.

Flow-based ML NIDS. ML Defender operates exclusively at the flow level, extracting 28 behavioral features per flow without examining payload content — ensuring that payload encryption does not compromise detection capability.

eBPF and XDP-based network monitoring. Recent work has explored eBPF/XDP as a foundation for high-performance network security. ML Defender follows the XDP-capture-plus-userspace-inference architecture: XDP provides zero-copy packet capture before the kernel networking stack, while all ML inference executes in userspace at sub-microsecond latency. To our knowledge, open-source projects combining eBPF capture with userspace ML inference remain rare; most production-grade alternatives either offload inference to the cloud or require GPU hardware.

Embedded inference. Commercial EDR vendors (CrowdStrike, SentinelOne, Carbon Black) implement proprietary embedded ML pipelines, but neither their architectures nor their training data are publicly available. ML Defender demonstrates that embedded Random Forest inference achieves sub-microsecond latency without requiring GPU acceleration or cloud offload.

Human-AI collaborative engineering. Structured methodologies for multi-model peer review have not, to our knowledge, been formally documented in the systems security literature. Section 6 describes the *Consejo de Sabios* methodology.

3 Threat Model

3.1 Adversary Capabilities

The system assumes an adversary capable of launching attacks from external hosts or compromised internal machines, including: botnet C2 communication, DDoS (including amplification attacks), ransomware propagation via SMB lateral movement, and network reconnaissance.

3.2 Adversary Limitations

The system assumes the attacker cannot: fully mimic benign traffic distributions across all monitored features simultaneously; compromise the ML Defender host; manipulate feature extraction; or tamper with HMAC-SHA256 protected event logs.

3.3 Detection Surface

Detection occurs at the network flow level. ML Defender focuses on connection frequency, port diversity, TCP flag distributions, packet inter-arrival timing, flow burst behavior, and destination diversity — features detectable even when payloads are encrypted.

3.4 Out-of-Scope Attacks

Application-layer attacks generating statistically normal traffic; low-and-slow attacks within benign statistical distributions; insider threats; attacks targeting the ML Defender host itself. Future work in Section 11 addresses some of these through expanded detection vectors (FEAT-NET-1, FEAT-AUTH-1).

4 Architecture

4.1 Pipeline Overview

Six components connected via ZeroMQ and Protocol Buffers:

1. **eBPF/XDP Sniffer** — zero-copy packet capture at the XDP hook before kernel networking stack.
2. **Ring Consumer** — reconstructs bidirectional flow state via ShardedFlowManager, computes 28 of 40 planned features. Eleven features use `MISSING_FEATURE_SENTINEL = -9999.0f`.
3. **Fast Detector** — rule-based heuristic engine. FPR on purely benign traffic: 6.61% (§8.6).
4. **ML Detector** — embedded Random Forest across four threat classes. Three classifiers (Ransomware, DDoS, Traffic) in pure C++20; Internal threats via ONNX Runtime. Latency: 0.24–1.06 μs.
5. **RAG Subsystem** (`rag-ingester` + `rag-security`) — rag-ingester ingests detection events into FAISS + SQLite. rag-security exposes a natural language query interface

via local TinyLlama — deliberately local to avoid exfiltrating live network metadata to cloud-hosted LLMs.

6. **Firewall ACL Agent** — applies ipset/iptables rules on confirmed detection events.

Figure 1 illustrates the end-to-end flow from packet capture to automated firewall response.

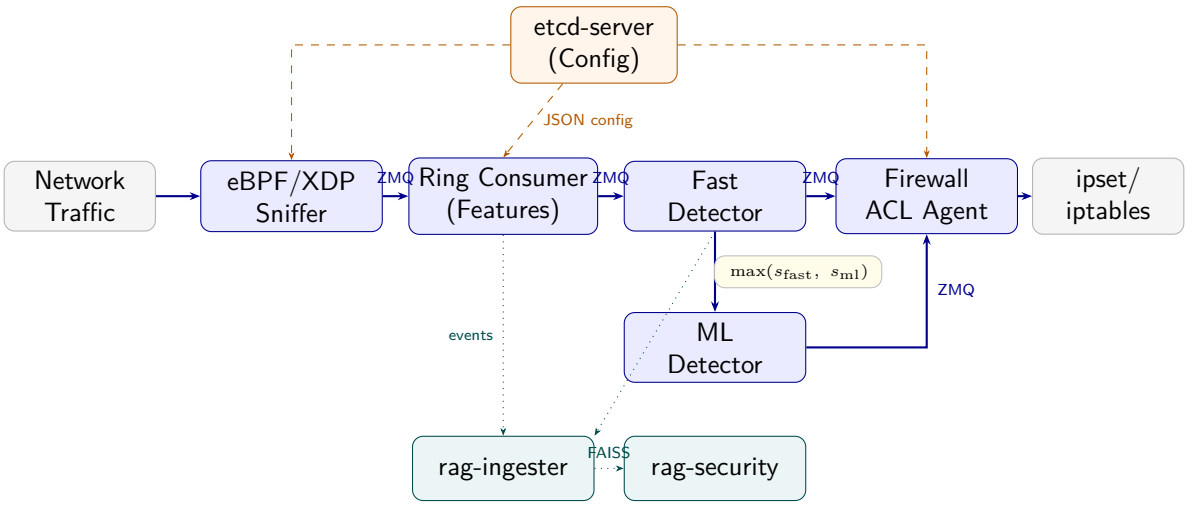

Figure 1: ML Defender end-to-end pipeline. Six components communicate over ZeroMQ with Protocol Buffers serialization and ChaCha20-Poly1305 encryption. `etcd-server` distributes JSON configuration at startup. The dual-score *Maximum Threat Wins* policy (Eq. 1) selects the arithmetic maximum of Fast Detector and ML Detector scores; the RAG subsystem provides semantic observability over the live event stream.

4.2 Three Functions of the Engine

Function 1 — Real-time detection, classification, and response. End-to-end from packet capture to firewall block in milliseconds.

Function 2 — Reliable dataset generation. ML Detector and Firewall ACL Agent produce structured HMAC-verified CSV logs continuously, forming the basis for retraining future ensemble models.

Function 3 — Real-time observability (the digital microscope). The RAG subsystem provides a semantic query interface over the live event stream.

4.3 Dual-Score Detection

$$score_{\text{final}} = \max(score_{\text{fast}},\ score_{\text{ml}}) \tag{1}$$

This is the *arithmetic maximum* over two continuous scores in $[0, 1]$, not a logical OR over binary decisions. Both detectors output continuous confidence scores; the maximum operator selects the higher-confidence assessment. The ML Detector's primary role is false positive suppression: the Fast Detector generates FPR = 6.61% on purely benign traffic; the ML Detector reduces real production blocks to zero on the same corpus (approximately 500-fold reduction). Alternative AND-based consensus policies are planned (ADR-007, §11).

4.4 Transport and Serialization

ZeroMQ for asynchronous message passing; Protocol Buffers (proto3) for serialization; etcd for configuration distribution; ChaCha20-Poly1305 for inter-component encryption (selected for performance on hardware without AES-NI, including ARMv8); HMAC-SHA256 for CSV log integrity.

4.5 Deployment Model

The reference deployment targets a single bare-metal Linux node (kernel ≥ 5.8) with two NICs: one connected to the monitored network segment, one to the management network. The full pipeline runs on commodity hardware costing approximately 150–200 USD. Validated in Vagrant/VirtualBox; bare-metal characterization is Future Work (§11).

Figure 2 illustrates a representative high-availability deployment in a hospital environment, with a primary and a warm-standby node protecting two internal network segments.

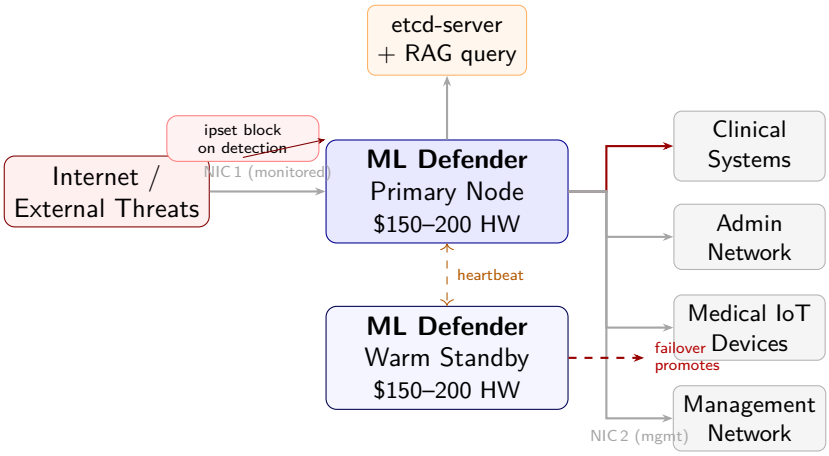

Figure 2: Representative high-availability deployment of ML Defender in a hospital environment. A primary node monitors all inbound and inter-segment traffic via NIC 1 (monitored); a warm-standby node receives heartbeat signals and promotes automatically on primary failure. Both nodes run the full six-component pipeline on commodity hardware. `etcd-server` and the RAG query interface are co-located on the primary node. **Firewall ACL Agent topology:** scored Protocol Buffers events flow from `ml-detector` to one or more `firewall-acl-agent` instances, which apply `ipset/iptables` rules at the relevant boundary. A single gateway node requires one agent; each independent segment with its own firewall requires its own agent instance. **etcd-server** is shown in single-node mode (current implementation); a distributed HA etcd cluster is in the project backlog. Peer-to-peer seed negotiation is under design.

4.6 Service Coordination (etcd-server)

Fulfills component registration, JSON config distribution, and ChaCha20 seed provision. ChaCha20 seed distribution via etcd is **not recommended for production use** — peer-to-peer seed negotiation is under design. etcd is currently deployed in *single-node* mode (see Figure 2); a distributed HA etcd cluster (3- or 5-node consensus) is in the project backlog and will be required for true high-availability deployments. The single-node constraint is the only architectural dependency preventing fully symmetric active/standby failover.

4.7 Fast Detector Heuristics (DEBT-FD-001)

Four heuristics in a 10-second sliding window:

- External IP velocity: `THRESHOLD_IP_VELOCITY = 5`
- SMB connection count: `THRESHOLD_SMB_CONNS = 3`
- Port diversity: `THRESHOLD_PORT_SCAN = 10`
- TCP RST ratio: `THRESHOLD_RST_RATIO = 0.20`

All thresholds are compile-time `constexpr` constants — DEBT-FD-001, scheduled for JSON migration in PHASE 2 (ADR-006).

5 Implementation

5.1 Language and Build System

C++20. CMake with four build profiles: production, debug, TSAN, ASAN.

5.2 eBPF/XDP Packet Capture

XDP hook attachment before SKB allocation. Zero-copy. All parameters loaded from `sniffer/config/sniffer.` at startup.

5.3 Feature Extraction

ShardedFlowManager partitions flow tracking across shards, eliminating the ISSUE-003 thread-isolation bug (89 of 142 features silently lost in the original single-threaded FlowManager). 28 of 40 features computed; 11 use `MISSING_FEATURE_SENTINEL = -9999.0f`.

Sentinel value selection rationale. Split-domain analysis confirmed all Random Forest thresholds lie within $[0.0, 5.1]$. -9999.0f lies entirely outside this domain, routing deterministically to the left child of every split — non-informative, not influencing classification. `quiet_NaN()` was rejected (undefined comparison behavior across compilers); 0.5f was rejected (within domain, spurious classification influence). One feature uses 0.5f as a semantic sentinel for TCP half-open state (deliberate, within-domain meaningful default).

5.4 Embedded Random Forest and Synthetic Training Data

Three classifiers (Ransomware, DDoS, Traffic) compiled as C++20 data structures. Internal threats classifier via ONNX Runtime — deliberate to measure latency differential and footprint trade-off. Models trained with scikit-learn, transpiled to C++20 via custom script; zero Python or external runtime dependency.

Training/evaluation separation: all classifiers trained on synthetic data; CTU-13 Neris held out entirely for evaluation.

Synthetic Dataset Methodology. Informed by CTU-13 (flow statistics), CIC-IDS2017 (protocol ratios), and MAWI (benign traffic). Four bias mitigation strategies: IP Space Partitioning (10.0.0.0/8 training vs 147.32.0.0/16 evaluation), Temporal Decoupling, Protocol Ratio Calibration, Feature Boundary Validation.

Known Limitations. Reflects attack patterns circa 2011–2017. Temporal dynamics of real C2 channels simplified to fixed-window aggregates. Zero-day variants may evade classification.

5.5 Cryptographic Transport

ChaCha20-Poly1305 + LZ4 + HMAC-SHA256. Crypto-transport layer extracted as an independent library.

5.6 Plugin Architecture (ADR-023, ADR-024)

ML Defender implements a multi-layer plugin architecture (ADR-023) enabling runtime extension of pipeline components without modifying core detection logic. Plugins are loaded via `dlopen/dlsym` with a pure C ABI (`PLUGIN_API_VERSION = 1`), providing ABI stability across compiler versions and ensuring binary compatibility with future releases.

MessageContext and Graceful Degradation. PHASE 2a (DAY 105) introduces `MessageContext` — a typed C struct passed to the optional `plugin_process_message()` symbol. Plugins that do not export this symbol are silently skipped (Graceful Degradation Policy D1), preserving backward compatibility with PHASE 1 plugins that implement only `plugin_process_packet()`. Post-invocation validation (D8) performs a byte-wise snapshot comparison of all read-only fields, detecting any plugin attempt to modify ownership-restricted data. The trust boundary is explicit: plugins are treated as untrusted code (D7); the final blocking decision remains exclusively in the core (ADR-012).

Threat Model Integration (ADR-023 D9). The plugin system's Trusted Computing Base (TCB) is limited to `PluginLoader` and `CryptoTransport`. Plugin code has no access to cryptographic keys, seed material, or internal transport state. The `MLD_DEV_MODE` escape hatch is restricted to `Debug` builds compiled with `MLD_ALLOW_DEV_MODE` (D10), ensuring fail-closed behavior in all production builds.

Gate TEST-INTEG-4a. Integration of the plugin loader into `firewall-acl-agent` (PLUGIN-LOADER-FW) was validated by gate `TEST-INTEG-4a`: `plugin_process_message()` invoked on a live `MessageContext`, post-invocation invariants verified, `result_code = 0` confirmed, and `CryptoTransport` decryption path verified unmodified by diff.

Forward Compatibility: Dynamic Group Key Agreement (ADR-024). The `MessageContext` struct reserves 60 bytes (`reserved[60]`) for forward compatibility with ADR-024, which specifies Dynamic Group Key Agreement using the Noise Protocol Framework (`Noise_IKpsk3` or `Noise_KK` pattern). ADR-024 design is approved; open questions OQ-5 through OQ-8 (static

key revocation, session continuity during rotation, replay protection, and ARMv8 performance characterization) are under resolution. Implementation is scheduled post-arXiv submission.

5.7 Trace Correlation

`trace_id = SHA-256(src_ip, dst_ip, canonical_attack_type, timestamp_bucket)`. TraceIdPolicy: ransomware=60s, DDoS=10s, SSH brute=30s. Validated: 46/46 unit tests passing.

5.8 Testing and Validation

70 tests total: crypto (3), etcd-hmac (12), ml-detector (9), trace_id (46). 100% pass rate. Integration gate `TEST-INTEG-4a` (plugin `MessageContext` invocation, D8 post-invocation validation) confirmed on DAY 105. F1 ground truth persisted to `docs/experiments/f1_replay_log.csv`.

5.9 Development Environment

Vagrant/VirtualBox. Host: MacBook Pro Intel Core i9 2.5 GHz, 32 GB DDR4. Guests: `defender` (Ubuntu 24.04 LTS, Linux 6.x, dual vNIC) + `client` (traffic generator).

6 The Consejo de Sabios: A Methodology for Human-AI Collaborative Engineering

6.1 Motivation

The development of ML Defender spans kernel programming (eBPF/XDP), distributed systems (ZeroMQ, etcd), cryptography, machine learning, systems security, and academic writing. The author operated as an independent researcher without institutional affiliation, without a team, and without access to a formal peer review process. The solution was to construct one.

6.2 The Methodology

Seven large language models served as co-reviewers across all development phases: **Claude** (Anthropic), **Grok** (xAI), **ChatGPT** (OpenAI), **DeepSeek**, **Qwen** (Alibaba), **Gemini** (Google), **Parallel.ai**.

These models were engaged as intellectual peers — presented with architectural proposals, asked to identify failure modes, invited to challenge assumptions. The methodology operates on a principle of *structured disagreement*: the Council does not average outputs. It introduces adversarial validation as a mechanism to expose hidden assumptions and validate architectural decisions. Convergent recommendations increase confidence; divergent recommendations mandate deeper investigation before any decision is closed.

6.3 How It Worked in Practice

Convergent recommendations across models increased confidence. Divergent recommendations triggered deeper investigation. Decisions such as the -9999.0f sentinel value, the trace_id

correlation architecture, and the ShardedFlowManager design emerged from or were refined through these dialogues.

6.4 Test Driven Hardening

Test-Driven Hardening (TDH) is proposed as a methodology for building security-critical distributed systems in which correctness properties are protocol-level, not component-level, and therefore undetectable by static analysis or isolated unit tests. A standalone reference repository documenting the methodology and representative case studies is maintained at https://github.com/alonsoir/test-driven-hardening.

TDH methodology: (1) Witness — characterize anomalous behavior. (2) Demonstration test — produce a minimal test that fails and exposes the mechanism. (3) Independent solution — each model proposes a fix against the shared contract. (4) Human arbitration — author selects, weighing correctness, maintainability, and coherence.

TDH differs from TDD: the test is written *before the fix*, as a proof of failure — not before the feature, as a specification.

6.5 HKDF Context Symmetry: A Pedagogical Case Study in Test-Driven Hardening

A subtle but critical defect discovered during the cryptographic integration sprint of *ML Defender* illustrates a class of correctness errors that are invisible to static analysis yet detectable through end-to-end protocol testing. We document it here as a concrete instance of the Test-Driven Hardening methodology described in §6.

The defect. The `CryptoTransport` layer derives symmetric keys using HKDF-SHA256 [7] with a context string (*info* parameter) that encodes the identity of the communicating parties. The original implementation parameterised this context by *component name*:

Listing 1: Incorrect HKDF context — component-scoped

```
// WRONG: each component uses its own name as HKDF context.
// Sniffer derives K_sniffer; ml-detector derives K_mldetector.
// K_sniffer != K_mldetector -> MAC authentication failure on every channel.
const std::string ctx = "ml-defender:" + component_name
                      + ":" + version;
```

Because each component used its own name as the HKDF context, transmitter and receiver independently derived *distinct* subkeys — causing MAC authentication failure on every cross-component channel. The defect was latent in unit tests, which exercised each component in isolation using loopback (same context on both sides), where MAC verification trivially succeeds. It only manifested when two components with *different names* attempted to communicate end-to-end.

To make the causal chain explicit: an incorrect context string causes the transmitter and receiver to derive *distinct* subkeys, resulting in MAC verification failure at the receiver. A correct,

symmetric context string produces *identical* derived keys on both sides, enabling successful authentication. This class of error is undetectable without end-to-end protocol validation.

The correct parameterisation encodes *directionality*:

Listing 2: Correct HKDF context — channel-scoped

```
// RIGHT: tx and rx derive distinct subkeys
const std::string ctx_tx = "ml-defender:" + component_name
                           + ":" + version + ":tx";
const std::string ctx_rx = "ml-defender:" + component_name
                           + ":" + version + ":rx";
```

The canonical definitions were consolidated in `crypto_transport/include/crypto_transport/contexts.h` providing a single source of truth for all six pipeline components. This design avoids semantic overloading of data structures across layers — a common source of subtle security and correctness bugs, as demonstrated by the defect described above.

Why static analysis cannot catch this. Both the erroneous and correct context strings share the type `std::string`. The C++ type system offers no mechanism to distinguish a *transmit context* from a *receive context* at the type level without a dedicated wrapper type — an abstraction whose overhead is not warranted in a resource-constrained deployment target. The compiler, sanitisers (ASan, TSan), and `clang-tidy` all accepted both formulations without warning. This is a *semantic* correctness property, not a syntactic one.

Detection via TEST-INTEG-3. The defect was exposed by `TEST-INTEG-3`, an integration test exercising the full sniffer → ml-detector cryptographic channel with an intentional regression: the test temporarily restores the component-scoped context on one side and asserts that decryption fails with a MAC authentication error. Under the correct implementation, this regression triggers `std::terminate()` via the fail-closed handler; under the defective implementation, decryption succeeds — and the test fails accordingly.

The key insight is that `TEST-INTEG-3` does not test encryption or decryption in isolation. It tests the *protocol invariant*: that a message encrypted on the transmit path is decryptable only on the corresponding receive path, and that any violation of this invariant is immediately fatal. No unit test of `CryptoTransport` in isolation could have established this property.

Lessons for cryptographic software. This case study yields three transferable observations:

1. **Cryptographic correctness is a protocol property.** Components that are individually correct may be collectively insecure. End-to-end tests exercising full message round-trips across trust boundaries are the minimal viable test surface for authenticated encryption schemes.

2. **Intentional regression tests are first-class citizens.** A test that asserts failure under a known-bad configuration is as valuable as one that asserts success under a known-good one. The TDH methodology mandates both polarities.

3. **Single source of truth for cryptographic parameters.** Distributing context strings across six `main.cpp` files created a silent coordination hazard. Centralising them in a dedicated header eliminated an entire class of latent divergence.

The defect was discovered on Day 97 of the development timeline, three days after `CryptoTransport` was integrated across all pipeline components. Its detection required no external audit: the TDH test suite caught it during routine regression testing on the `feature/bare-metal-arxiv` branch (now superseded by `feature/plugin-crypto`).

6.6 Attribution

The author remained the final arbiter of all decisions; all empirical validation was performed by the human author. The seven models are credited as intellectual co-contributors in the project's commit history, ADRs, and this paper. In the context of arXiv submission, the models are acknowledged in §13; formal co-authorship conventions for AI systems remain an evolving area, and this paper adopts the most transparent available approach consistent with emerging ACM and IEEE norms on AI contribution disclosure.

6.7 Why This Matters

The Consejo de Sabios democratizes access to deep technical peer review for independent researchers without institutional affiliation — parallel to ML Defender's mission of democratizing enterprise-grade security for organizations without enterprise budgets.

6.8 Limitations

Models cannot run code or observe system behavior directly. Models can hallucinate — several incorrect technical suggestions were identified and rejected. The methodology augments human judgment; it does not replace it.

7 Formal System Model

7.1 Network Traffic Representation

Packets $P = \{p_1, \ldots, p_n\}$ grouped into bidirectional flows by canonical five-tuple $(src, dst, sport, dport, proto)$ via ShardedFlowManager $\Phi : P \to F$. Flows are bidirectional five-tuple aggregates; behavioral features are computed over a 10-second sliding window as configured in `sniffer.json`. Flows are closed after an inactivity timeout as defined by the sliding window expiry.

7.2 Feature Extraction

$$x_f = g(P_f, W), \quad x_f \in \mathbb{R}^{28} \tag{2}$$

7.3 Fast Detector

$$H(x_f) = \max_j h_j(x_f), \quad h_j(x_f) \in \{0, 1\} \tag{3}$$

7.4 Machine Learning Detector

$$M(x_f) = \frac{1}{T} \sum_{k=1}^{T} T_k(x_f) \tag{4}$$

7.5 Detection Policy

$$S(x_f) = \max(H(x_f),\ M(x_f)) \geq \theta \tag{5}$$

Thresholds: $\theta_{\text{ransomware}} = 0.85$, $\theta_{\text{DDoS}} = 0.90$, $\theta_{\text{traffic}} = 0.80$, $\theta_{\text{internal}} = 0.85$.

7.6 Response Function

$$R(f) = \begin{cases} \texttt{block}(src_f) & \text{if } S(x_f) \geq \theta \\ \texttt{allow} & \text{otherwise} \end{cases} \tag{6}$$

Blocking is applied per source IP. Blocks are currently persistent for the session duration; TTL-bounded blocks with configurable expiry are Future Work (§11).

7.7 System Determinism

Given identical packet input P, the pipeline produces identical outputs. Determinism is a prerequisite for the reproducibility guarantees described in Section 13.

8 Evaluation

8.1 Experimental Setup

Table 1: Experimental hardware and software specifications.

Parameter	**Value**
Host machine	Apple MacBook Pro (Intel Core i9, 2.5 GHz)
Host RAM	32 GB DDR4
Guest OS	Ubuntu 24.04 LTS (Linux kernel 6.x)
Virtualization	VirtualBox 7.2, bridged NIC mode
VM vCPUs	4 (estimated ∼2.4 GHz under VirtualBox)
VM RAM	4 GB (8 GB allocated for stress tests)
Network adapter	Paravirtualized (VirtIO)
Virt. overhead	Estimated 5–15% vs bare-metal
Pipeline config.	Full 6-component, dual-VM

Methodological note on metrics. `calculate_f1_neris.py` measures **Fast Detector** alerts (`[FAST ALERT]` in sniffer.log) against CTU-13 Neris ground truth IPs. The ML Detector's `attacks=N` statistic reports flows exceeding production thresholds and triggering firewall blocks. Both are reported; F1=0.9985 is the primary system detection metric.

8.2 Dataset

CTU-13 Neris. 19,135 total flows; **646 malicious flows** (host 147.32.84.165) constitute the TP ground truth. Captured 2011. Garcia et al. [4] document that the Neris botnet exhibits C2 communication patterns concentrated in a behavioral subset — specifically, flows matching IRC-based C2 channel signatures (connection bursts, SMB lateral movement, DNS query anomalies). The remaining flows represent background traffic on the infected host that do not exhibit malicious behavioral signatures and are therefore not counted as ground-truth positives. This distinction aligns with standard practice in NIDS evaluation [8, 12].

BigFlows. A CTU-13 derived dataset containing 40,467 flows and 791,615 packets, used for high-throughput replay experiments. 38,064 flows were processed by ml-detector. Treated conservatively as *probable benign* — no official ground truth exists for this CTU-13 scenario.

Synthetic training data. All classifiers trained exclusively on synthetic data; CTU-13 Neris held out entirely for evaluation.

8.3 Detection Performance

Table 2: Detection performance — Fast Detector pipeline on CTU-13 Neris.

Metric	Value
F1 Score	**0.9985**
Precision	0.9969
Recall	1.0000
True Positives	646
False Positives	2
False Negatives	0
True Negatives	12,075
Total events processed	12,723
FPR (Neris evaluation)	**0.0002%**
ML Detector high-conf. attacks	12
FPR Fast Detector (bigFlows)	6.61%
FP reduction (Fast → ML)	**≈500-fold**

F1=0.9985 stable across replay runs at total_events = 12,563 / 12,605 / 12,723 / 13,930.

8.4 False Positive Identification and Confusion Matrix

Table 3: False positive identification.

FP	Source	Destination	Classification
FP-1	192.168.56.1	224.0.0.251	mDNS multicast (VirtualBox host-only)
FP-2	192.168.56.1	192.168.56.255	Broadcast (VirtualBox host-only)

Both are VirtualBox host-only adapter artifacts. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified (§11).

Confusion matrix.

$$\text{Precision} = \frac{646}{648} = 0.9969, \quad \text{Recall} = \frac{646}{646} = 1.0000, \quad \text{F1} = 0.9985$$

8.5 Inference Latency

Table 4: Per-class inference latency.

Classifier	Implementation	Latency
DDoS	Embedded C++20	0.24 μs
Traffic	Embedded C++20	0.37 μs
Internal	ONNX Runtime	0.33 μs
Ransomware	Embedded C++20	1.06 μs

Latency values are mean inference times measured via `std::chrono::high_resolution_clock` around the `classify()` call within the ml-detector process, excluding inter-component ZeroMQ transport latency, over $N \geq 1{,}000$ inference calls per classifier during the CTU-13 Neris replay.

8.6 Ablation Study

Config A — Fast Detector Only: 2,517 FP on 38,064 bigFlows (FPR=6.61%), Recall=1.0000 on Neris.

Config B — ML Detector Only (inferred): 5 detections on 38,064 bigFlows, all below production thresholds (max confidence 68.97%) — 0 real blocks. Neris: attacks=12.

Config C — Dual (deployed): F1=0.9985, FPR=0.0002%. Only Config C fully validated end-to-end.

8.7 Interpretation and Comparison with State of the Art

F1=0.9985 and Recall=1.0000 reflect the statistical separability of the Neris botnet scenario, not a claim about arbitrary threats. The synthetic training successfully captured the Neris behavioral signature; generalization to other families is an open empirical question.

Table 5: Indicative comparison with representative NIDS approaches. *Caution: cross-paper comparison is indicative only — systems evaluated on different datasets, at different operating points, and under different experimental conditions. Direct experimental comparison was not performed.*

System	Dataset	F1	Latency	HW	Block	Notes
ML Defender	CTU-13 Neris	**0.9985**	0.24–1.06 μs	x86/ARM	Yes	Synthetic, dual-score
Kitsune [8]	Custom	∼0.982	∼50 μs	x86	No	Autoencoder ensemble
Suricata (rules)	Various	∼0.891	∼10 μs	x86	Yes	Signature+heuristics
RF [12]	CIC-IDS2017	∼0.943	∼200 μs	x86	No	Batch, not embedded
Snort	N/A	N/A	1–5 ms	x86	Yes	Rule-based, no ML

8.8 Ransomware Detection: Scope and Validation

What is validated. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the 2011 Neris botnet scenario: SMB lateral movement (ports 445/139 burst), high-rate connection attempts, DNS query anomalies.

What is not validated. Direct evaluation against post-2020 ransomware families (LockBit, BlackCat/ALPHV, Cl0p) has not been performed. Modern ransomware uses encrypted C2, adaptive beaconing with jitter, staged delivery — potentially different statistical signatures.

Future work. Direct evaluation against modern ransomware captures is the highest-priority evaluation expansion (§11), with Sebastian Garcia (CTU Prague) as potential collaborator.

8.9 Throughput Stress Test and Resource Utilization

Table 6: Stress test replay results — progressive Mbps escalation (DAY 87).

Run	Dataset	Req.	Actual	PPS	Packets	Failed	Errors
1	Neris	10 Mbps	9.60 Mbps	∼3,700	320,524	2,630†	0
2	Neris	25 Mbps	11.28 Mbps	∼4,300	320,524	2,630†	0
3	bigFlows	50 Mbps	34.68 Mbps	9–10K	791,615	0	0
4	bigFlows	100 Mbps (×3)	33.16 Mbps	9–11K	2,374,845	0	0

† Pre-existing pcap artifact — identical count at 10 and 25 Mbps, confirming origin in pcap structure, not pipeline saturation.

Table 7: Resource utilization during 100 Mbps bigFlows replay (DAY 87).

Component	CPU%	RES	MEM%	Notes
ml-detector	∼315–320%	164 MB	2.0%	3.2 cores; multi-threaded
sniffer	∼88–108%	55–57 MB	0.7%	XDP + ring consumer
firewall-acl	∼10–16%	101 MB	1.2%	Stable throughout
rag-ingester	∼12–20%	36–39 MB	0.5%	FAISS indexing
etcd-server	∼0.2%	10 MB	0.1%	Minimal
rag-security	idle	1.2 GB VIRT	15.0%‡	TinyLlama loaded
Total	∼65–73%	∼1,278 MB	—	∼23–30% idle

‡ rag-security's 1.2 GB virtual footprint is the TinyLlama model mapped at startup.

The throughput ceiling is VirtualBox NIC emulation (∼33–38 Mbps), not pipeline logic. Zero packet drops, zero deserialization errors across all 2,374,845 packets. Total pipeline RAM stable at ∼1.28 GB (±18 MB drift) throughout the 8-minute test.

9 Performance Model and Throughput Analysis

9.1 Pipeline Processing Model

Six stages operating concurrently via ZeroMQ; effective throughput bounded by the slowest stage:

$$T_{\max} = \frac{1}{\max_i(L_i)} \tag{7}$$

9.2 Inference Cost

$\mathcal{O}(T \cdot D)$ — simple threshold comparisons on compact C++20 data structures fitting in CPU cache.

9.3 Throughput: Theoretical Bound and Measured Ceiling

Theoretical upper bound (single-core, ML inference only):

$$T_{\text{theory}} \approx \frac{1}{0.5\ \mu\text{s}} = 2 \times 10^6 \text{ flows/sec} \tag{8}$$

Measured virtualized ceiling (DAY 87). The VirtIO NIC limits delivery to ∼33–38 Mbps (∼9,000–11,000 pps). The observed throughput is limited by the virtualized NIC, not the pipeline. At this ceiling: ml-detector sustains ∼3.2 cores; total pipeline RAM stable at ∼1.28 GB; system CPU ∼65–73% user, ∼23–30% idle.

9.4 Queue Stability

Stability requires $\lambda < \mu$. Post-replay drain behavior provides direct empirical evidence: sniffer CPU drops to near-zero while ml-detector continues draining at full throughput for several minutes — confirming the ZeroMQ buffer absorbed the burst cleanly. `errors=(deser:0, feat:0, inf:0)` confirmed at all observation points.

9.5 Scalability

Vertical: additional CPU cores → additional parallel flow-processing threads, linear throughput improvement up to capture bottleneck.

Horizontal: multiple ML Defender nodes across network segments, each processing local traffic independently while contributing to distributed dataset generation.

10 Limitations

10.1 Evaluation Scope. Single botnet scenario (CTU-13 Neris, 2011). Modern ransomware communication patterns may differ substantially. Generalizability not empirically established.

10.2 Incomplete Feature Set. 28 of 40 features implemented; 11 use −9999.0f sentinel. Impact of missing 12 features on broader traffic distributions unknown.

10.3 Synthetic Training Data. Reflects patterns circa 2011–2017. Zero-day variants may evade classification.

10.4 Fast Detector False Positive Rate. FPR=6.61% on bigFlows under current JSON-configurable thresholds (Path B, active since DAY 80). An earlier compile-time baseline (Path A, DAY 13–79, DEBT-FD-001) produced FPR=76.8% on the same corpus; DAY 80 JSON threshold migration achieved a 12× improvement.

10.5 Cryptographic Seed Distribution. ChaCha20 seed via etcd not recommended for production. Peer-to-peer negotiation under design.

10.6 Single-Node Deployment. Distributed deployment not validated experimentally.

10.7 etcd High Availability. Single-node; failure disrupts registration and config distribution. HA planned.

10.8 RAG Index Capacity. FAISS event store finite capacity not mathematically characterized.

10.9 Virtualization Overhead. All latency and throughput measurements obtained under VirtualBox. Reported latencies are conservative upper bounds; reported throughput (∼33–38 Mbps) is a conservative lower bound imposed by NIC emulation. Bare-metal characterization remains future work (§11).

10.10 BigFlows Ground Truth. Treated as *probable benign*, not *confirmed benign*. FPR=0.0002% is a lower bound.

10.11 Threats to Validity.

1. **Seed compromise.** If `seed.bin` is exfiltrated, an adversary can derive all component subkeys and forge authenticated messages. Mitigations: strict file permissions (0600), in-memory zeroing via `explicit_bzero`, and planned peer-to-peer key agreement (ADR-024).

2. **Single-host key assumption.** All six components currently share a common seed family without distributed key agreement. Isolation between deployments relies on distinct seed provisioning per instance (ADR-021); formal distributed key agreement is post-arXiv work (ADR-024).

3. **No hardware root of trust.** The current design does not leverage TPM or secure enclave primitives. Physical access to the host enables seed extraction. Hardware-anchored key storage is identified as a future hardening target.

11 Future Work

11.1 Evaluation Corpus Expansion (P0). CTU-13 additional scenarios, modern ransomware captures, volumetric DDoS datasets. Contact initiated with Sebastian Garcia (CTU Prague) as potential arXiv endorser and collaborator.

11.2 Feature Completion and Importance Analysis. Complete remaining 12 real features. Feature importance analysis (Gini impurity reduction per feature from embedded RF).

11.3 Fast Detector JSON Migration (DEBT-FD-001). Scheduled for PHASE 2.

11.4 Consensus Detection Policy (ADR-007). AND-based policy requiring simultaneous agreement before autonomous blocking — JSON-configurable for operator selection.

11.5 Response Strategy Expansion. TTL-bounded blocks, traffic quarantine, honeypot redirection.

11.6 Cryptographic Hardening. Peer-to-peer ChaCha20 seed negotiation; etcd HA.

11.7 Runtime Reconfiguration (Hot-Reload). JSON parameter updates without restart.

11.8 Graph Attack Engine (ENT-2). Live attack graphs from CSV event data; STIX 2.1 export.

11.9 Fleet-Wide Observability. Fine-tuned LLM over anonymized aggregated data from all deployed nodes.

11.10 Test Driven Hardening as Independent Research. TDH generalizability, scaling with model capability, and measurable outcomes vs single-model consultation.

11.11 Bare-Metal Throughput Stress Testing (P1). Progressive tcpreplay at 100/250/500/1000 Mbps on physical NIC; CPU/memory/packet-drop monitoring per component. All throughput figures in this paper should be treated as conservative lower bounds.

11.12 Infiltration Vector Coverage. ML Defender currently detects ransomware *propagation*. Future work addresses *infiltration*: FEAT-NET-1 (DNS anomaly/DGA detection), FEAT-NET-2 (threat intelligence feeds), FEAT-AUTH-1 (authentication log ingestion), FEAT-AUTH-2 (behavioral auth anomaly), FEAT-EDR-1 (lightweight endpoint agent).

11.13 Enterprise Features. ENT-1 (Federated Threat Intelligence), ENT-2 (Attack Graph Generation), ENT-3 (P2P Seed Distribution), ENT-4 (Hot-Reload). MIT-licensed core remains free.

11.14 Ethical Deployment in Healthcare Environments. Not certified for FDA 21 CFR Part 820 or EU MDR 2017/745. Phased deployment: (1) 30-day monitoring mode with human approval; (2) TTL-bounded blocks with SOC notification; (3) full autonomous blocking after site-specific FPR validation.

11.15 Federated Distributed Intelligence (ENT-1 Extension). The current architecture processes all detection locally at each deployed node. A planned federated extension — not yet designed or implemented, and explicitly scoped to post-pipeline-completion milestones — would enable a coordinated fleet of ML Defender nodes to contribute anonymized telemetry to a central aggregation tier for continuous model retraining, with updated classifiers distributed back to the fleet as signed plugins (ADR-025). Figure 3 illustrates the intended architecture; implementation is contingent on full pipeline stabilization and will be reflected in a future revision of this paper.

12 Conclusion

This paper presented ML Defender (aRGus NDR), an open-source network detection and response system built in C++20, designed from first principles to protect the organizations that need protection most and can afford it least.

Evaluated against real botnet traffic from the CTU-13 Neris dataset — traffic the system had never seen during training — ML Defender achieves F1 = 0.9985, Precision = 0.9969, and Recall = 1.0000. The two false positives are fully identified: an mDNS multicast frame and a broadcast frame, both generated by the VirtualBox host-only NIC. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified; bare-metal characterization remains Future Work (§11). The Fast Detector alone produces a 6.61% FPR on purely benign traffic; the ML Detector reduces this to zero real production blocks on the same

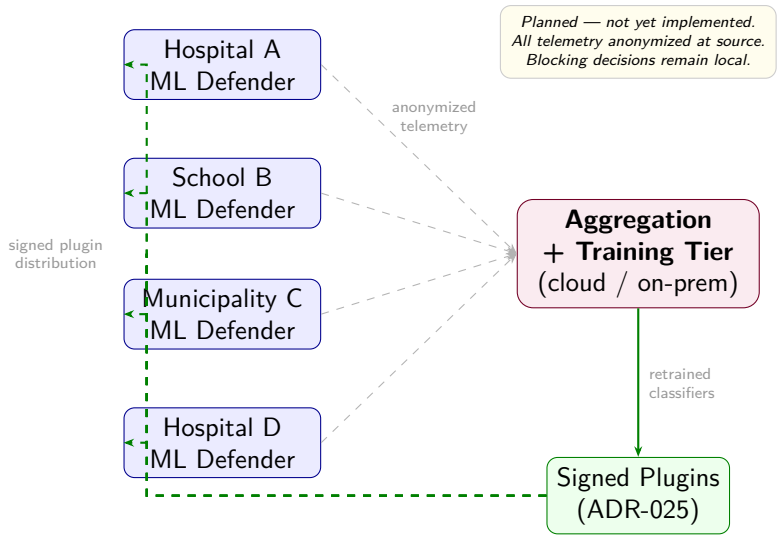

Figure 3: Planned federated distributed intelligence architecture (*not yet implemented*). Each deployed ML Defender node operates fully autonomously; anonymized flow-level telemetry is contributed to a central aggregation tier for periodic retraining. Updated classifiers are packaged as Ed25519-signed plugins (ADR-025) and distributed to the fleet. All detection and response decisions remain local to each node; no live network metadata leaves the deployment site. This architecture is a post-pipeline-stabilization milestone; a future revision of this paper will document the implementation.

corpus. End-to-end inference latency ranges from 0.24 μs to 1.06 μs per flow, entirely in-process, on commodity hardware costing approximately 150–200 USD.

Under progressive load testing, the pipeline processes up to ∼34–38 Mbps — the ceiling imposed by VirtualBox NIC emulation — with zero packet drops, zero errors, and stable RAM (∼1.28 GB) across 2.37 million replayed packets. The bottleneck is the hypervisor, not the software.

These results cover one botnet scenario from 2011. They demonstrate architectural viability and effective detection within that scope. The limitations in Section 10 are the honest documentation of a system that does what it claims to do within the boundaries where it has been tested.

ML Defender was built because a friend lost his business to ransomware, and because a hospital in Barcelona was paralyzed by an attack on the same day its author was leaving a regional hospital in Extremadura — the hospital that had diagnosed his porphyria and saved his life. The design constraints that followed from those two events were not abstract: open-source, commodity hardware, real-time response. Not a forensic tool. Not a dashboard. A system that acts.

The *Consejo de Sabios* represents a new mode of engineering: one that gives a single independent researcher access to the kind of multi-expert peer review that has historically required institutional affiliation, research groups, and geographic proximity to centers of expertise. The *Test Driven Hardening* methodology that emerged from this collaboration has proven effective against some of the most subtle concurrency and memory bugs in the codebase.

Both contributions — the technical system and the collaborative methodology — are acts of democratization. The parallel is not accidental. It is the point.

For the hospital in Extremadura. For the friend whose business was broken by ransomware. For every small organization that has been told, implicitly or explicitly, that enterprise-grade security is not for them.

It is now.

13 Reproducibility and Artifact Availability

All experiments are reproducible from the public repository: https://github.com/alonsoir/argus

Dataset Setup.

```
# CTU-13 Neris (scenario 10) -- required for F1 validation
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
botnet-capture-20110810-neris.pcap -O /vagrant/datasets/ctu13/neris.pcap
# Mirror: https://www.stratosphereips.org/datasets-ctu13

# CTU-13 bigFlows -- required for stress test
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
bigFlows.pcap -O /vagrant/datasets/ctu13/bigFlows.pcap
```

F1 Evaluation.

```
make pipeline-stop && make logs-lab-clean && make pipeline-start && sleep 15
make test-replay-neris
python3 scripts/calculate_f1_neris.py \
  /vagrant/logs/lab/sniffer.log --total-events 19135
```

Stress Test.

```
vagrant ssh client -c "sudo tcpreplay -i eth1 --mbps=100 --loop=3 \
  /vagrant/datasets/ctu13/bigFlows.pcap"
```

Determinism. Classifiers embedded in binary; no stochastic inference components; feature extraction deterministic. F1=0.9985 confirmed stable at total_events = 12,563 / 12,605 / 12,723 / 13,930.

Acknowledgments

This work was carried out independently, without institutional funding, without a research group, and without affiliation to any university or laboratory.

The author engaged seven large language models — Claude (Anthropic), Grok (xAI), ChatGPT (OpenAI), DeepSeek, Qwen (Alibaba), Gemini (Google), and Parallel.ai — as structured intellectual collaborators across 88 days of continuous development. Their contributions to architectural decisions, failure mode identification, debugging, test scenarios, and paper review are

documented in the project's commit history and ADRs. The *Test Driven Hardening* methodology documented in Section 6 emerged directly from this collaboration.

This paper discloses AI involvement openly and precisely. The author remained the final arbiter of all decisions; all empirical validation was performed by the human author.

The author wishes to acknowledge the friend whose business was destroyed by ransomware — who will recognize himself in these pages. And the Hospital Universitario de Badajoz, Extremadura, whose physicians diagnosed a rare disease and gave the author the years needed to build something worth building. Thank you Pedro and Agustín.

This work is dedicated to every hospital, school, and small organization that has ever been told that enterprise-grade security is beyond their reach.

References

[1] Blake Anderson and David McGrew. Identifying encrypted malware traffic with contextual flow data. In *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 35–46, 2016.

[2] Black Fog. The state of ransomware 2025. https://www.blackfog.com/, 2025.

[3] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2): 1153–1176, 2016.

[4] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. An empirical comparison of botnet detection methods. *Computers & Security*, 45:100–123, 2014.

[5] IBM Security. Cost of a data breach report 2025, 2025.

[6] INCIBE-CERT. Ciberataque ransomware paraliza actividad del Hospital Clínic de Barcelona. https://www.incibe.es/, 2023.

[7] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, Internet Engineering Task Force, May 2010.

[8] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Network and Distributed Systems Security Symposium (NDSS)*, 2018.

[9] Open Information Security Foundation. Suricata open source IDS/IPS/NSM engine. https://suricata.io/, 2010.

[10] André Pinto et al. Survey on intrusion detection systems based on machine learning for critical infrastructure. *Sensors*, 23(5):2415, 2023.

[11] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, pages 229–238, 1999.

[12] Iman Sharafaldin, Ali Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.